

Connect to the workspace

API documentation and integration guide | Application API 3.0.0 | Tourism envelope 2.0 | Verified 27 September 2026

One runtime across four interfaces

Atlas Office exposes 69 registered skills through its local browser workspace, HTTP API, per-skill command line and company MCP server. Each registered skill has a manifest, JSON input schema, embedded input example, CLI runner and SKILL.md instructions. The nine tourism skills are included in the 69, not additional to them.

The API is a single-user local prototype. It binds only to 127.0.0.1, with port 8765 by default. It has no account login, bearer-token authentication or per-employee access control. A person_id selects a configured colleague; it is not an authenticated identity.

Start and discover

Interactive API documentation is available at /docs. /api/openapi.json is an alias of /openapi.json. The generated OpenAPI 3.1.0 document contains a concrete run route and input schema for each registered skill. Runtime examples in this guide were checked against the implementation; generated IDs and timestamps vary.

```
# Run from chengdu-tourism-office; Python 3.11+
python -m pip install -r requirements.txt
python app.py --mode api --port 8765

# Optional isolated local state
python app.py --data-dir ./demo-state

# Discovery endpoints
GET http://127.0.0.1:8765/api/health
GET http://127.0.0.1:8765/api/skills
GET http://127.0.0.1:8765/api/skills/budget-variance
GET http://127.0.0.1:8765/openapi.json
```

Request boundaries

- Send JSON objects for POST requests, normally with Content-Type: application/json. Successful requests use HTTP 200; file downloads return bytes.
- The Host header must be 127.0.0.1:<port> or localhost:<port>. Browser mutations reject a mismatched Origin or Sec-Fetch-Site: cross-site. CLI clients can omit Origin.
- Most JSON request bodies are limited to 1,000,000 bytes. File uploads allow a 21,000,000-byte JSON envelope and at most 15,000,000 decoded bytes.
- Local state defaults to output/ and includes SQLite run, audit and conversation stores plus generated files. --data-dir or ATLAS_DATA_DIR selects another state directory.

Sources: app.py: Handler and main | scripts/workspace_api.py: specification/get/post | scripts/skill_runtime.py: Runtime.catalog

Run a skill and inspect its result

Example request

inputs is required and validated against the selected manifest's JSON Schema. person_id defaults to atlas. Other colleagues must be available and explicitly assigned the skill. Unknown top-level run fields are rejected. IDs use letters, digits, underscore or hyphen, up to 90 characters.

An idempotency_key returns the existing run for an identical skill, input and person. Reusing it for different parameters produces an error.

```
POST /api/skills/budget-variance/run
{
  "inputs": {
    "currency": "CNY",
    "items": [{
      "category": "Marketing",
      "budget": 10000, "actual": 11500
    }]
  },
  "person_id": "atlas",
  "idempotency_key": "DOCS-001"
}
```

Response shape

Abbreviated response; input echoes the full submitted payload. GET /api/runs/{id} retrieves the persisted record. Inspect status and output even after HTTP 200: a skill failure is saved as a run result.

```
{
  "id": "run_<generated-id>",
  "skill_id": "budget-variance",
  "person_id": "atlas",
  "status": "completed",
  "input": {"currency": "CNY", "items": [...]},
  "output": {
    "summary": "Positive variance means overspending.",
    "currency": "CNY",
    "items": [{
      "category": "Marketing",
      "budget": 10000, "actual": 11500,
      "variance": 1500.0, "variance_percent": 15.0
    }]
  },
  "created_at": "<ISO timestamp>",
  "updated_at": "<ISO timestamp>",
  "idempotency_key": "DOCS-001"
}
```

Run lifecycle and approval

Approval and rejection accept a JSON object such as {}. Runtime state transitions prevent repeated execution of the same run. A changed Google connection or MCP configuration invalidates the prepared connection fingerprint. For an uncertain external result, inspect the destination before preparing another run.

Effect / state	Behavior
Local or read action	queued -> running -> completed, or failed
remote_write / reviewed_write	awaiting_approval; no action executes yet
POST /api/runs/{id}/approve	Executes the prepared action; failure becomes needs_attention
POST /api/runs/{id}/reject	Cancels an awaiting_approval run
completed	Execution finished; inspect business output and artifacts

Sources: scripts/skill_runtime.py: submit/approve/reject/_execute | skills/budget-variance/skill.json | scripts/business_tools.py: variance handler

Workspace endpoint reference

HTTP routes

Method and route	Purpose / input
GET /api/health	{status: ok, version: 3.0.0, skills: 69}
GET /api/skills	Manifest catalog, schemas, examples, effects and paths
GET /api/skills/{id}	One manifest plus its SKILL.md instructions
POST /api/skills/{id}/run	inputs; optional person_id and idempotency_key
GET /api/runs or /api/runs/{id}	Latest 200 runs, or one stored run
POST /api/runs/{id}/approve or /reject	Execute or cancel the exact pending run
GET /api/files	Latest 300 files: id, name, size, created_at
POST /api/files	name and content_base64
GET /api/files/{id}/download	File bytes with download filename header
GET /api/records?kind=...&q=...	Filter up to 1,000 recent local records
POST /api/records/{id}	Update using the record's original skill input schema
GET /api/audit	Latest 300 recorded audit events
GET /api/company	Current company and editable staff roster
POST /api/company	Save a validated company object
GET /api/templates	Available general and tourism templates
POST /api/company/template	{template: general} or {template: tourism}; backs up old roster
GET /api/connections	Public Google, Telegram and MCP connection status
POST /api/mcp/discover	{server_id: configured-id}; cache tool schemas
GET /api/config	Public AI configuration, without the credential
POST /api/config/key	{api_key: <new secret>}; server-side key configuration
GET or POST /api/settings	Read or save tourism autonomy and delivery rules

File upload example

Supported extensions are .csv, .xlsx, .docx, .pptx, .pdf, .txt, .md, .json, .eml and .ics. Store the returned ID and pass it to file-based skills. Generated artifacts use the same file registry. Uploading a file does not execute it or send it externally.

```
POST /api/files
{"name": "note.txt", "content_base64": "SGVsbG8="}

# Response shape
{
  "id": "file_<generated-id>",
  "name": "note.txt", "size": 5,
  "url": "/api/files/file_<generated-id>/download"
}
```

Sources: scripts/workspace_api.py: get/post | app.py: Handler.do_POST | scripts/skill_runtime.py: files/records/audit

The nine tourism skills

Callable IDs and stage inputs

Pass a preceding stage's data object to downstream inputs, not its whole envelope. Every stage accepts a JSON object and is callable through `/api/skills/{id}/run`, the company MCP `run_skill` tool, and the tourism CLI.

Skill ID	Input and principal output
inquiry-intake	message; optional explicit trip fields -> normalized inquiry, source provenance, missing fields
flight-search	inquiry; optional snapshot -> eligible and rejected CNY group flight estimates
tour-search	inquiry; optional catalog -> activities ranked by interests, price and capacity
option-comparison	inquiry; optional flight, activities, priority -> feasible options and cost breakdown
hierarchy-router	optional people and task_kind -> lead, duty owners, reporting routes and capability gaps
workload-splitter	inquiry, option, hierarchy -> owned task graph, dates, dependencies and approval gates
schedule-builder	inquiry, activities; optional flight -> dated schedule and unscheduled activities
execution-controller	tasks, events -> accepted evidence, rejected events, readiness and escalations
outcome-reporter	inquiry, option, hierarchy, execution -> hold/review decision, evidence register and email draft

Tourism envelope

Abbreviated stage output. Tourism statuses are `ok`, `needs_input`, `blocked` and `escalated`. Through the shared runtime this envelope is nested inside the run's output. An outer completed status means the handler finished; it does not mean the trip is approved or a blocked stage succeeded.

Known integration limitation: five tourism handlers retain legacy inner skill labels. Use the requested route and outer `skill_id` as the authoritative stage identity. Invocation routing and stage data are correctly mapped.

```
{
  "schema_version": "2.0",
  "request_id": "EXAMPLE-RFQ",
  "skill": "inquiry-intake",
  "status": "ok",
  "data": {"group_size": 30, "budget_cny": 200000},
  "issues": [],
  "trace": ["customer:message", "rule:intake-v2"]
}
```

Important semantics

- Intake requires a message of at least 15 characters, positive whole group size and CNY budget, duration of 1-21 days, and a non-past ISO travel date. Fields can be extracted from the sentence or supplied explicitly.
- Synthetic flight and activity fixtures are unconfirmed. A supplied flight snapshot must normalize currency to CNY; group capacity, route and dates are checked. Schedule output includes arrival and departure across `duration_days + 1` dates.
- Execution accepts completion only from the assigned owner with `evidence_ref`, completed dependencies and any required gate flag. This validates simulated evidence, not real supplier confirmation.

Sources: `scripts/tourism_core.py`; `SKILLS/HANDLERS` and stage functions | `shared/message-schema.json` | `skills/*/skill.json`

Conversations, proposals and cases

Start a reviewed tourism workflow

message must contain 1-6,000 characters after trimming. Omit conversation_id to create a conversation, or provide it to continue. Optional request_id links a saved case. The response is {conversation, case, proposal}, with case or proposal omitted when absent.

With run_workflow true, saved company policy controls proposal execution. The client auto_execute flag cannot bypass that policy. The separate /api/simulate route runs the lower-level synthetic pipeline and defaults auto_execute to true; use false for a planning-only simulation. It is not a substitute for the reviewed /api/chat flow.

```
# First select the tourism template
POST /api/company/template
{"template": "tourism"}

POST /api/chat
{
  "message": "RFQ for a Chengdu food and tea tour.",
  "group_size": 30, "duration_days": 3,
  "travel_start": "2026-10-25", "budget_cny": 200000,
  "person_id": "atlas",
  "run_workflow": true
}
```

Workflow routes

Method and route	Contract
GET /api/conversations	Saved titles, previews, timestamps and linked request IDs
POST /api/conversations/new	Exactly title and person_id; creates an empty conversation
GET /api/conversation/{id}	Conversation, ordered messages and proposal
GET /api/conversation/{id}/events	Stored per-person progress events
POST /api/chat	message plus optional conversation_id, person_id, request_id, run_workflow
POST /api/review	conversation_id, action, comment
POST /api/simulate	message; optional trip fields, auto_execute, preferred_option
GET /api/case/{id}	Stored case trace
POST /api/event	request_id, task_id, action; incident can include reason
POST /api/reassign	request_id, duty, person_id; enforces skill and capacity
POST /api/option	request_id, option_id; chooses a feasible quote
POST /api/agent	person_id, question; optional request_id for grounded answers
GET /api/search?q=...	Search locally indexed cases
GET /download/{filename}	Allowed SIM- case DOCX, PPTX or trace JSON

Review and revision

Supported review actions are approve, request_changes, escalate, resolve, resubmit and approve_email. Comments allow up to 3,000 characters; returning, escalating or resolving requires an explanation. The response refreshes conversation, case and proposal.

Case fingerprints bind approval to the reviewed request, quote, schedule and assignments. Changing an option or owner invalidates affected evidence and requires renewed review. Comments alone do not edit the case. Email delivery is separately governed by saved opt-in settings, approved recipients and server configuration.

Sources: app.py: chat/review/simulate/_update_case/Handler | scripts/governance.py | references/api.md

CLI, MCP and operational limits

Command-line interfaces

For the general runner, --input - reads stdin and --data-dir selects the workspace. A remote or reviewed write remains awaiting_approval; CLI and MCP submission do not approve it. For host configuration, use absolute Python and server-file paths with stdio transport.

```
# Standalone tourism stage; @ loads a JSON file
python scripts/tourism_office.py run inquiry-intake '@examples/rfq.json'

# Full synthetic case with generated report artifacts
python scripts/tourism_office.py case '@examples/rfq.json' --out ./demo-output

# Any registered skill uses its shared-runtime runner
python skills/budget-variance/scripts/run.py --input budget.json

# Local MCP servers; launch through an MCP host
python server/company_mcp.py
python server/tourism_mcp.py
```

MCP tool surfaces

The company server's run_skill can invoke all nine tourism stages as well as the other registered skills. The specialized tourism server offers six convenience tools. Its send tool additionally checks stored readiness, a human-held approval code, the recipient allowlist and SMTP settings before STARTTLS delivery. Demo .test addresses are rejected.

Outbound MCP connections require an enabled server, an explicit allowed_tools list and successful schema discovery. Remote HTTP connections require HTTPS, except loopback HTTP; bearer credentials come from server environment variables. Connection templates are not proof of a working integration.

Server	Tools and behavior
company_mcp.py	list_skills(category), read_skill(skill_id), run_skill(skill_id, arguments, person_id, idempotency_key), list_runs(), list_files(), search_knowledge(query), get_company()
tourism_mcp.py	simulate_rfq(message, request_id, customer_email), compare_flights(group_size, travel_start, duration_days, origin_airport), find_tours(group_size, interests, accessible), search_prior_cases(query, limit), preview_report_email(request_id), send_approved_report_email(request_id, approval_code)

Errors and deployment boundary

No reservations or payments are made by the tourism simulation. OpenAI, Google, Telegram, SMTP and external MCP features depend on separately configured services. Public AI configuration never returns the API credential. Exposing this workspace to other users requires identity, authorization and an HTTPS application server; local origin checks do not supply those controls.

Result	Caller action
400 {error: ...}	Correct JSON, required fields, schema, ID or transition
403 {error: ...}	Check local origin, assigned skill or review permission
404 {error: ...}	Check skill, file, case, conversation or run ID
500 {error: ...}	Generic server failure; inspect local configuration
200 with failed / needs_attention	Read output.error; execution outcome is stored
Tourism needs_input / blocked / escalated	Inspect output.issues and complete missing conditions

Sources: scripts/tourism_office.py | scripts/skill_runtime.py: cli | server/company_mcp.py | server/tourism_mcp.py | scripts/mcp_bridge.py | app.py: Handler

Agent orchestration and execution statistics

One request, bounded model-led execution

POST /api/chat with message and optional conversation_id/person_id runs the selected company assistant. With a configured API key, Atlas exposes find_skills, read_skill, read_skills, set_plan, run_skill, list_workspace_files, ask_user and web search to the configured Responses model. The model chooses the sequence and inputs. There is no hard-coded launch pipeline.

The turn is bounded to 48 model requests, 80 function-tool calls and 600 seconds checked between operations. A single in-flight model request may extend past the time check. Read skill contracts before execution. Planned runs must match their step assignee and skill and may execute only after prerequisites complete. Qualified staff are enforced by the runtime. External and reviewed writes remain pending for explicit review.

Polling and persistence

Route / field	Contract
GET /api/conversation/{id}/agent-run	Latest persisted agent execution object, or null before the first run.
GET /api/conversation/{id}/events	Timestamped progress events.
conversation.agent_run	The same snapshot included when reading or completing a conversation.
steps[]	id, title, skill_id, person_id, depends_on, status; optional run_id and error.
calls[]	id, tool, status, started_at, duration_ms; optional skill_id/person_id/step_id/run_id/error.

Measured fields and completion states

statistics contains model_requests, tool_calls, skills_executed, skills_completed, skills_failed, skills_pending, distinct_skills, delegated_people, artifacts, elapsed_ms and input/output/total_tokens. Tokens sum provider usage across requests, including repeated context; they are null if usage is unavailable. They are not a cost estimate. Tool-call counts refer to function tools; built-in web search activity is separately exposed in progress events.

Run states are running, completed, needs_input, awaiting_approval, needs_attention and budget_exhausted. Offline directory mode requests a configured connection and produces no skill runs. Failed or pending work cannot complete a planned dependency. A completed skill means its local handler finished, not that a real-world business outcome occurred.

Sources: Application: scripts/company_chat.py, scripts/agent_execution.py, scripts/conversation_store.py, app.py | OpenAI function calling: <https://developers.openai.com/api/docs/guides/function-calling>

Complete skill parameter reference

How to call every skill

The following 69 entries cover every registered skill in this submission. The first line states purpose and effect; each entry then lists input parameters, their types and required flags, nested fields, and the exact manifest and endpoint for full schema and example data.

A star (*) means required in the containing object. A field without a star is optional. `array<T>` indicates a JSON array whose elements have type `T`. A union such as `string/number` allows either type. An empty string is still a supplied value where permitted. All numbers must be finite.

Read each entry's JSON example from `skills/<id>/skill.json` at `#/example` and its complete schema at `#/input_schema`. `GET /api/skills/<id>` returns these plus SKILL.md instructions. All paths in this appendix are relative to `chengdu-tourism-office/`.

All 69 manifests, embedded examples, SKILL.md files and CLI runners were checked. All embedded examples satisfy their declared schemas. Some examples contain placeholder file, run or record IDs, and connected-service examples need their corresponding account configuration before execution. Schema validity does not imply a completed external operation.

Shared request wrapper

Only inputs is required in this wrapper. All entries below describe the object inside inputs. For MCP, pass the same object as `run_skill`'s arguments. For CLI, save it as JSON and use `python skills/<id>/scripts/run.py --input <path>`.

The nine tourism manifests declare generic object schemas, so their handler-level parameters are documented explicitly from `tourism_core.py`. The remaining skills use detailed schema properties. Their full string-length limits and any additional schema restrictions remain visible in the referenced manifest.

```
POST /api/skills/<id>/run
{
  "inputs": { "skill-specific fields": "values" },
  "person_id": "atlas",
  "idempotency_key": "optional-unique-key"
}
```

Effect and review policy

These effects are manifest declarations enforced by the shared runtime. External MCP calls are conservatively marked `remote_write` even when a selected tool appears read-only. Tourism autonomy settings do not bypass general skill approvals.

Effect	Runtime behavior
local	Runs immediately on local data; may create files or records
remote_read	Reads a configured external service without a review stop
remote_write	Prepares <code>awaiting_approval</code> ; review is required to execute
reviewed_write	Local file/finance change is held for review before execution

Sources: `scripts/skill_runtime.py` | `scripts/workspace_api.py` | `scripts/tourism_core.py` | `skills/*/skill.json`

Skill reference 01-05

booking-confirm | Simulate booking confirmation

Record a clearly simulated customer booking decision without reserving a supplier, taking payment or sending email. Effect: local.

Parameters: customer*: string - Customer name; nonempty, max 200 chars; product*: string - Tourism product; nonempty, max 300 chars; travel_date*: string - Travel date YYYY-MM-DD; format date; quote_cny*: number - Quoted customer amount CNY; >0; supplier_option*: string - Selected indicative supplier option; nonempty, max 300 chars.

Reference: skills/booking-confirm/skill.json (#/input_schema, #/example). GET /api/skills/booking-confirm.

budget-variance | Budget variance

Compare budget with actual spending, including percentage variance. Effect: local.

Parameters: currency*: string - Currency; nonempty; items*: array<object> - Budget lines; items 0-500.

items[] members: category*: string - Category; nonempty; budget*: number - Budget; >=0; actual*: number - Actual; >=0.

Reference: skills/budget-variance/skill.json (#/input_schema, #/example). GET /api/skills/budget-variance.

calendar-create | Create Google Calendar event

Create an event in your primary calendar without emailing attendees. Effect: remote_write.

Parameters: title*: string - Event title; nonempty; start*: string - Start, ISO datetime with offset; nonempty; end*: string - End, ISO datetime with offset; nonempty; description*: string - Agenda; nonempty.

Reference: skills/calendar-create/skill.json (#/input_schema, #/example). GET /api/skills/calendar-create.

calendar-event | Calendar invitation

Create an .ics event for Outlook, Apple Calendar or Google Calendar. Effect: local.

Parameters: title*: string - Event title; nonempty; start*: string - Start, ISO date and time with offset; nonempty; end*: string - End, ISO date and time with offset; nonempty; description*: string - Agenda; nonempty.

Reference: skills/calendar-event/skill.json (#/input_schema, #/example). GET /api/skills/calendar-event.

calendar-invite | Send test calendar invitation

After review, create a Google Calendar event and invite only the configured demo test inbox. Effect: remote_write.

Parameters: title*: string - title; nonempty, max 200 chars; start*: string - start; min 20 chars; end*: string - end; min 20 chars; description*: string - description; nonempty; attendee*: string - attendee; format email.

Reference: skills/calendar-invite/skill.json (#/input_schema, #/example). GET /api/skills/calendar-invite.

Sources: skills/booking-confirm/skill.json | skills/budget-variance/skill.json | skills/calendar-create/skill.json | skills/calendar-event/skill.json | skills/calendar-invite/skill.json

Skill reference 06-10

calendar-list | Read Google Calendar

List upcoming calendar events within a supplied date range. Effect: remote_read.

Parameters: start*: string - Range start, ISO datetime with offset; nonempty; end*: string - Range end, ISO datetime with offset; nonempty.

Reference: skills/calendar-list/skill.json (#/input_schema, #/example). GET /api/skills/calendar-list.

campaign-plan | Campaign plan

Create a campaign brief with channels, budget and measurable objectives. Effect: local.

Parameters: title*: string - Campaign; nonempty; audience*: string - Audience; nonempty; objective*: string - Objective; nonempty; channels*: array<string> - Channels; items 0-500; budget*: number - Budget; >=0; currency*: string - Currency; nonempty; success_measure*: string - Success measure; nonempty.

Reference: skills/campaign-plan/skill.json (#/input_schema, #/example). GET /api/skills/campaign-plan.

contract-checklist | Contract review checklist

Organize supplied contract clauses, evidence and review questions. Effect: local.

Parameters: title*: string - Review title; nonempty; items*: array<object> - Checks; items 0-500.

items[] members: requirement*: string - Requirement or clause; nonempty; evidence*: string - Evidence; status*: string - Status; one of met/gap/unknown; owner*: string - Owner; nonempty.

Reference: skills/contract-checklist/skill.json (#/input_schema, #/example). GET /api/skills/contract-checklist.

crm-pipeline | Sales pipeline

Calculate weighted pipeline value and summarize opportunities by stage. Effect: local.

Parameters: currency*: string - Currency; nonempty; deals*: array<object> - Opportunities; items 0-500.

deals[] members: customer*: string - Customer; nonempty; stage*: string - Stage; nonempty; value*: number - Value; >=0; probability*: number - Probability, 0 to 1; >=0, <=1.

Reference: skills/crm-pipeline/skill.json (#/input_schema, #/example). GET /api/skills/crm-pipeline.

csv-clean | Clean tabular data

Trim CSV data, optionally remove exact duplicate rows, and profile the result. Effect: local.

Parameters: file_id*: string - Uploaded CSV ID; nonempty; deduplicate*: boolean - Remove duplicate rows.

Reference: skills/csv-clean/skill.json (#/input_schema, #/example). GET /api/skills/csv-clean.

Sources: skills/calendar-list/skill.json | skills/campaign-plan/skill.json | skills/contract-checklist/skill.json | skills/crm-pipeline/skill.json | skills/csv-clean/skill.json

Skill reference 11-15

customer-followup | Customer follow-up register

Record a customer's request, owner, promised next action and due date so work continues after the initial quote. Effect: local.

Parameters: title*: string - title; nonempty, max 150 chars; customer*: string - customer; nonempty, max 200 chars; request*: string - request; nonempty; owner*: string - owner; nonempty, max 100 chars; status*: string - status; one of inquiry/proposal_sent/awaiting_customer/confirmed_simulated/needs_attention/closed; next_action*: string - next action; nonempty, max 500 chars; due_date*: string - due date; format date; reference_id*: string - reference id; max 100 chars.

Reference: skills/customer-followup/skill.json (#/input_schema, #/example). GET /api/skills/customer-followup.

decision-log | Decision log

Save a decision with its rationale, owner and follow-up date. Effect: local.

Parameters: title*: string - Decision; nonempty; rationale*: string - Rationale; nonempty; owner*: string - Decision owner; nonempty; review_date*: string - Review date; nonempty.

Reference: skills/decision-log/skill.json (#/input_schema, #/example). GET /api/skills/decision-log.

docs-create | Create Google document

Create a Google document and insert the supplied text. Effect: remote_write.

Parameters: title*: string - Title; nonempty; body*: string - Document text; nonempty.

Reference: skills/docs-create/skill.json (#/input_schema, #/example). GET /api/skills/docs-create.

docs-read | Read Google Docs

Read the structured content of a Google document. Effect: remote_read.

Parameters: document_id*: string - Document ID; nonempty.

Reference: skills/docs-read/skill.json (#/input_schema, #/example). GET /api/skills/docs-read.

document-create | Word document

Create a formatted Word document with headings and editable content. Effect: local.

Parameters: title*: string - Document title; nonempty; sections*: array<object> - Sections; items 0-50.

sections[] members: heading*: string - Heading; nonempty; body*: string - Body; nonempty.

Reference: skills/document-create/skill.json (#/input_schema, #/example). GET /api/skills/document-create.

Sources: skills/customer-followup/skill.json | skills/decision-log/skill.json | skills/docs-create/skill.json | skills/docs-read/skill.json | skills/document-create/skill.json

Skill reference 16-20

drive-search | Search Google Drive

Find files by name in the connected Google Drive account. Effect: remote_read.

Parameters: query*: string - File name contains; nonempty; limit*: integer - Result limit; >=1, <=100.

Reference: skills/drive-search/skill.json (#/input_schema, #/example). GET /api/skills/drive-search.

drive-upload | Upload to Google Drive

Upload a selected local workspace file to Google Drive. Effect: remote_write.

Parameters: file_id*: string - Workspace file ID; nonempty; folder_id*: string - Destination folder ID; nonempty.

Reference: skills/drive-upload/skill.json (#/input_schema, #/example). GET /api/skills/drive-upload.

email-draft | Email draft

Create an editable .eml email with a subject, recipients and body. Effect: local.

Parameters: to*: string - Recipient email; nonempty; subject*: string - Subject; nonempty; body*: string - Message; nonempty.

Reference: skills/email-draft/skill.json (#/input_schema, #/example). GET /api/skills/email-draft.

execution-controller | Execution Controller

Run the existing synthetic tourism execution controller stage. Effect: local.

tasks*: nonempty array<object>. tasks[]: id*, owner_id, depends_on* (task-ID array), gate (flag name or null), route_from_lead (optional reporting path). Task IDs must be unique; dependencies must exist. request_id: string - trace identifier.

events: ordered array<object> (default []). events[]: task_id, action (complete, incident or resolve_incident); complete/resolve_incident require the assigned actor_id and evidence_ref. Completion also requires true for the task's dynamic gate flag, when present.

Incident events may carry reason. Dependencies must already be evidenced; duplicates, unknown tasks, missing evidence and out-of-order events are rejected in output.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/execution-controller/skill.json (#/input_schema, #/example). GET /api/skills/execution-controller.

expense-report | Expense report

Sum expenses by category and flag missing receipts for review. Effect: local.

Parameters: employee*: string - Employee; nonempty; currency*: string - Currency; nonempty; expenses*: array<object> - Expenses; items 0-500.

expenses[] members: description*: string - Description; nonempty; category*: string - Category; nonempty; amount*: number - Amount; >=0; receipt*: boolean - Receipt attached.

Reference: skills/expense-report/skill.json (#/input_schema, #/example). GET /api/skills/expense-report.

Sources: skills/drive-search/skill.json | skills/drive-upload/skill.json | skills/email-draft/skill.json | skills/execution-controller/skill.json | skills/expense-report/skill.json

Skill reference 21-25

finance-forecast | Monthly finance forecast

Forecast monthly customers, revenue, costs and operating profit from dated assumptions, with an Excel schedule. Effect: local.

Parameters: title*: string - title; nonempty, max 100 chars; currency*: string - currency; min 3 chars, max 3 chars; opening_customers*: integer - opening customers; >=0; periods*: array<object> - periods; items 1-24.

periods[] members: month*: string - month; pattern ^[0-9]{4}-(0[1-9]|1[0-2])\$; new_customers*: integer - new customers; >=0; churn*: integer - churn; >=0; revenue_per_customer*: number - revenue per customer; >=0; variable_cost_per_customer*: number - variable cost per customer; >=0; fixed_cost*: number - fixed cost; >=0.

Reference: skills/finance-forecast/skill.json (#/input_schema, #/example). GET /api/skills/finance-forecast.

finance-posting | Review booking finance and invoice draft

After a simulated confirmation, update customer income and supplier outflow projections and prepare a draft invoice, gated by human review. Effect: reviewed_write.

Parameters: booking_id*: string - Simulated booking record ID; nonempty; file_id*: string - Monthly finance workbook ID; nonempty; category*: string - Expense category; one of Flights/Hotels/Guides/Transport/Activities/Marketing/Customer care; committed_cny*: number - Simulated committed cost CNY; >0.

Reference: skills/finance-posting/skill.json (#/input_schema, #/example). GET /api/skills/finance-posting.

flight-search | Flight Search

Run the existing synthetic tourism flight search stage. Effect: local.

inquiry*: object - normalized trip. inquiry.group_size*, duration_days*: positive whole values; inquiry.travel_start*: ISO date. inquiry.origin_airport: string (default SIN); destination_airport: string (default CTU). request_id: string - trace identifier.

snapshot: object - optional replacement for bundled synthetic fixture; currency must be CNY and offers must be an array. Optional source and observed_at identify provenance.

snapshot.offers[]: id, origin, destination, depart_date, return_date, seats, fare_cny_per_person, tax_cny_per_person, stops, duration_minutes. Route/date/capacity/fare checks reject unsuitable offers; stops and duration sort eligible offers. Use the full fixture shape in resources/flight-offers.example.json.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/flight-search/skill.json (#/input_schema, #/example). GET /api/skills/flight-search.

gmail-draft | Create Gmail draft

Create an unsent message in Gmail after reviewing its contents. Effect: remote_write.

Parameters: to*: string - Recipient; nonempty; subject*: string - Subject; nonempty; body*: string - Message; nonempty.

Reference: skills/gmail-draft/skill.json (#/input_schema, #/example). GET /api/skills/gmail-draft.

gmail-search | Search Gmail

Search Gmail messages and return subject, sender and snippet. Effect: remote_read.

Parameters: query*: string - Gmail search query; nonempty; limit*: integer - Result limit; >=1, <=20.

Reference: skills/gmail-search/skill.json (#/input_schema, #/example). GET /api/skills/gmail-search.

Sources: skills/finance-forecast/skill.json | skills/finance-posting/skill.json | skills/flight-search/skill.json | skills/gmail-draft/skill.json | skills/gmail-search/skill.json

Skill reference 26-30

gmail-send | Send Gmail message

Send the exact reviewed recipient, subject and message through Gmail. Effect: remote_write.

Parameters: to*: string - Recipient; nonempty; subject*: string - Subject; nonempty; body*: string - Message; nonempty.

Reference: skills/gmail-send/skill.json (#/input_schema, #/example). GET /api/skills/gmail-send.

hierarchy-router | Hierarchy Router

Run the existing synthetic tourism hierarchy router stage. Effect: local.

people: array<object> - optional roster; defaults to bundled tourism staff. task_kind: string - inquiry favors the product manager; other values favor sales. request_id: string - trace identifier.

people[]: id, name, role, skills (array), reports_to (staff ID or null), capacity_hours and assigned_hours (nonnegative numeric), available (boolean); follows is an optional staff-ID array. IDs must be unique with one top manager, valid acyclic reporting links and skill/capacity matches.

Required roster structure is enforced by the handler, not a detailed manifest schema. Use resources/tourism-company.json as the complete roster example.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/hierarchy-router/skill.json (#/input_schema, #/example). GET /api/skills/hierarchy-router.

hiring-scorecard | Interview scorecard

Calculate a weighted interview score from job-related criteria and recorded evidence. Effect: local.

Parameters: candidate*: string - Candidate reference; nonempty; criteria*: array<object> - Criteria; items 0-500.

criteria[] members: criterion*: string - Job-related criterion; nonempty; weight*: number - Weight; >=0; score*: number - Score, 0 to 5; >=0, <=5; evidence*: string - Evidence; nonempty.

Reference: skills/hiring-scorecard/skill.json (#/input_schema, #/example). GET /api/skills/hiring-scorecard.

hr-onboarding | Employee onboarding

Create a dated onboarding checklist with accountable owners. Effect: local.

Parameters: employee*: string - Employee; nonempty; role*: string - Role; nonempty; start*: string - Start date; nonempty; manager*: string - Manager; nonempty.

Reference: skills/hr-onboarding/skill.json (#/input_schema, #/example). GET /api/skills/hr-onboarding.

incident-report | Incident report

Record an incident timeline, impact, owner and recovery actions. Effect: local.

Parameters: title*: string - Incident; nonempty; owner*: string - Incident owner; nonempty; impact*: string - Impact; nonempty; timeline*: string - Timeline; nonempty; actions*: array<object> - Actions; items 0-500.

actions[] members: task*: string - Task; nonempty; owner*: string - Owner; nonempty; due*: string - Due date; nonempty.

Reference: skills/incident-report/skill.json (#/input_schema, #/example). GET /api/skills/incident-report.

Sources: skills/gmail-send/skill.json | skills/hierarchy-router/skill.json | skills/hiring-scorecard/skill.json | skills/hr-onboarding/skill.json | skills/incident-report/skill.json

Skill reference 31-35

inquiry-intake | Inquiry Intake

Run the existing synthetic tourism inquiry intake stage. Effect: local.

message*: string - RFQ or inquiry text, at least 15 trimmed characters. request_id: string - trace identifier (default rfq-001).

group_size, duration_days, budget_cny: positive whole values, supplied explicitly or extracted from message; duration 1-21 days. travel_start: string - non-past YYYY-MM-DD date, explicit or extracted. These four business facts are required for status ok.

type: string - rfq or inquiry, otherwise inferred. origin_airport: string - uppercase three-letter code (default SIN).

customer_email: string/null - optional draft recipient. customer_name: value - optional retained customer label. Interests are extracted from message; destination is CTU.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/inquiry-intake/skill.json (#/input_schema, #/example). GET /api/skills/inquiry-intake.

inventory-reorder | Inventory reorder

Calculate reorder points and suggested quantities using demand, lead time and safety stock. Effect: local.

Parameters: items*: array<object> - Stock items; items 0-500.

items[] members: sku*: string - SKU; nonempty; on_hand*: number - On hand; >=0; on_order*: number - On order; >=0; daily_demand*: number - Daily demand; >=0; lead_days*: number - Lead time in days; >=0; safety_stock*: number - Safety stock; >=0; target_days*: number - Target days of cover; >=0.

Reference: skills/inventory-reorder/skill.json (#/input_schema, #/example). GET /api/skills/inventory-reorder.

invoice-create | Invoice draft

Calculate line totals, tax and amount due, then create an Excel invoice draft. Effect: local.

Parameters: customer*: string - Customer; nonempty; currency*: string - Currency; nonempty; tax_rate*: number - Tax rate, 0 to 1; >=0, <=1; items*: array<object> - Line items; items 0-500.

items[] members: description*: string - Description; nonempty; quantity*: number - Quantity; >=0; unit_price*: number - Unit price; >=0.

Reference: skills/invoice-create/skill.json (#/input_schema, #/example). GET /api/skills/invoice-create.

knowledge-save | Save company knowledge

Store a searchable company note with its source and owner. Effect: local.

Parameters: title*: string - Title; nonempty; content*: string - Content; nonempty; source*: string - Source; nonempty; owner*: string - Owner; nonempty.

Reference: skills/knowledge-save/skill.json (#/input_schema, #/example). GET /api/skills/knowledge-save.

knowledge-search | Search company knowledge

Search the local knowledge register and return matching source notes. Effect: local.

Parameters: query*: string - Search terms; nonempty.

Reference: skills/knowledge-search/skill.json (#/input_schema, #/example). GET /api/skills/knowledge-search.

Sources: skills/inquiry-intake/skill.json | skills/inventory-reorder/skill.json | skills/invoice-create/skill.json | skills/knowledge-save/skill.json | skills/knowledge-search/skill.json

Skill reference 36-40

leave-request | Leave request

Calculate requested weekdays and save a leave request for manager review. Effect: local.

Parameters: employee*: string - Employee; nonempty; start*: string - Start date; nonempty; end*: string - End date; nonempty; manager*: string - Manager; nonempty.

Reference: skills/leave-request/skill.json (#/input_schema, #/example). GET /api/skills/leave-request.

management-handoff | Management handoff

Summarize progress and blockers for a manager, with a Word brief and an email draft. Effect: local.

Parameters: title*: string - title; nonempty, max 100 chars; audience*: string - audience; nonempty, max 100 chars; items*: array<object> - items; items 1-100.

items[] members: work*: string - work; nonempty, max 150 chars; owner*: string - owner; nonempty, max 100 chars; status*: string - status; one of not_started/in_progress/blocked/done; due*: string - due; format date; next_step*: string - next step; nonempty, max 500 chars; blocker*: string - blocker; max 500 chars.

Reference: skills/management-handoff/skill.json (#/input_schema, #/example). GET /api/skills/management-handoff.

mcp-tool | Connected MCP tool

Prepare an allowlisted tool call on a configured MCP server. Effect: remote_write.

Parameters: server_id*: string - Connection ID; nonempty; tool*: string - Tool name; nonempty; arguments*: object - Tool arguments.

Reference: skills/mcp-tool/skill.json (#/input_schema, #/example). GET /api/skills/mcp-tool.

meeting-minutes | Meeting minutes

Turn supplied decisions and action items into structured meeting notes. Effect: local.

Parameters: title*: string - Meeting; nonempty; notes*: string - Notes; nonempty; decisions*: array<string> - Decisions; items 0-500; actions*: array<object> - Actions; items 0-500.

actions[] members: task*: string - Task; nonempty; owner*: string - Owner; nonempty; due*: string - Due date; nonempty.

Reference: skills/meeting-minutes/skill.json (#/input_schema, #/example). GET /api/skills/meeting-minutes.

option-comparison | Option Comparison

Run the existing synthetic tourism option comparison stage. Effect: local.

inquiry*: object with group_size*, duration_days*, budget_cny*: positive whole values and travel_start*: non-past ISO date. request_id: string - trace identifier.

flight: object with nonnegative group_total_cny, or omitted. activities: array<object>, each with nonnegative group_total_cny (default []). priority: string - balanced, price or experience (default balanced).

Uses resources/tourism-price-model.json for land cost, margin, capacity and lead-time rules. Returns blocked when no modeled option fits the supplied constraints.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/option-comparison/skill.json (#/input_schema, #/example). GET /api/skills/option-comparison.

Sources: skills/leave-request/skill.json | skills/management-handoff/skill.json | skills/mcp-tool/skill.json | skills/meeting-minutes/skill.json | skills/option-comparison/skill.json

Skill reference 41-45

outcome-reporter | Outcome Reporter

Run the existing synthetic tourism outcome reporter stage. Effect: local.

inquiry*, option*, hierarchy*, execution*: objects. request_id: string - trace identifier. inquiry uses customer_email, group_size and duration_days; option uses id and quote_cny; hierarchy uses task_lead and routes.

execution.client_draft_ready must be true for release readiness. execution.completed must be an object keyed by task IDs with evidence_ref values; execution.incidents must contain no open incident.

Expected evidence: INTAKE, DESIGN, SUPPLIER, PRICING, SAFETY and FINANCE, plus EXECUTIVE for quote_cny >= 100000. Missing evidence returns hold/blocked. Output is an unsent customer email draft and management evidence register.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/outcome-reporter/skill.json (#/input_schema, #/example). GET /api/skills/outcome-reporter.

pdf-create | PDF brief

Create a paginated, printable PDF brief from structured sections. Effect: local.

Parameters: title*: string - PDF title; nonempty; sections*: array<object> - Sections; items 0-50.

sections[] members: heading*: string - Heading; nonempty; body*: string - Body; nonempty.

Reference: skills/pdf-create/skill.json (#/input_schema, #/example). GET /api/skills/pdf-create.

pdf-extract | Read a PDF

Extract page text from an uploaded PDF for review and search. Effect: local.

Parameters: file_id*: string - Uploaded PDF ID; nonempty.

Reference: skills/pdf-extract/skill.json (#/input_schema, #/example). GET /api/skills/pdf-extract.

policy-checklist | Policy checklist

Assess supplied requirements against recorded evidence. Effect: local.

Parameters: title*: string - Review title; nonempty; items*: array<object> - Checks; items 0-500.

items[] members: requirement*: string - Requirement or clause; nonempty; evidence*: string - Evidence; status*: string - Status; one of met/gap/unknown; owner*: string - Owner; nonempty.

Reference: skills/policy-checklist/skill.json (#/input_schema, #/example). GET /api/skills/policy-checklist.

presentation-create | PowerPoint presentation

Build an editable presentation with a cover and concise content slides. Effect: local.

Parameters: title*: string - Presentation title; nonempty; slides*: array<object> - Slides; items 0-30.

slides[] members: title*: string - Slide title; nonempty; bullets*: array<string> - Key points; items 0-6.

Reference: skills/presentation-create/skill.json (#/input_schema, #/example). GET /api/skills/presentation-create.

Sources: skills/outcome-reporter/skill.json | skills/pdf-create/skill.json | skills/pdf-extract/skill.json | skills/policy-checklist/skill.json | skills/presentation-create/skill.json

Skill reference 46-50

procurement-compare | Supplier comparison

Rank suppliers by normalized cost, quality and delivery using explicit weights. Effect: local.

Parameters: cost_weight*: number - Cost weight; >=0; quality_weight*: number - Quality weight; >=0; delivery_weight*: number - Delivery weight; >=0; suppliers*: array<object> - Suppliers; items 0-500.

suppliers[] members: name*: string - Supplier; nonempty; cost*: number - Cost; >=0; quality*: number - Quality, 0 to 100; >=0, <=100; delivery_days*: number - Delivery days; >=0.

Reference: skills/procurement-compare/skill.json (#/input_schema, #/example). GET /api/skills/procurement-compare.

project-plan | Project plan

Create a dependency-aware project schedule and identify the critical delivery date. Effect: local.

Parameters: title*: string - Project; nonempty; start*: string - Start date, YYYY-MM-DD; nonempty; tasks*: array<object> - Tasks; items 0-500.

tasks[] members: id*: string - ID; nonempty; title*: string - Task; nonempty; owner*: string - Owner; nonempty; days*: integer - Duration in calendar days; >=1, <=365; depends_on*: array<string> - Dependencies; items 0-500.

Reference: skills/project-plan/skill.json (#/input_schema, #/example). GET /api/skills/project-plan.

proposal-package | Option proposal and slides

Compare business options under visible cost, impact and risk weights, then create a PowerPoint proposal and Word decision brief. Effect: local.

Parameters: title*: string - title; nonempty, max 120 chars; objective*: string - objective; nonempty, max 500 chars; audience*: string - audience; nonempty, max 100 chars; currency*: string - currency; min 3 chars, max 3 chars; weights*: object - weights; options*: array<object> - options; items 2-5.

weights members: impact*: number - impact; >=0; risk*: number - risk; >=0; cost*: number - cost; >=0.

options[] members: name*: string - name; nonempty, max 100 chars; cost*: number - cost; >=0; impact*: number - impact; >=1, <=5; risk*: number - risk; >=1, <=5; benefit*: string - benefit; nonempty, max 300 chars; concern*: string - concern; nonempty, max 300 chars.

Reference: skills/proposal-package/skill.json (#/input_schema, #/example). GET /api/skills/proposal-package.

quotation-package | Business quotation package

Calculate a general quotation with discount, tax and validity, then create a draft Word document and Excel price schedule. Effect: local.

Parameters: customer*: string - customer; nonempty, max 200 chars; currency*: string - currency; min 3 chars, max 3 chars; valid_until*: string - valid until; format date; discount_rate*: number - discount rate; >=0, <=1; tax_rate*: number - tax rate; >=0, <=1; terms*: string - terms; nonempty; items*: array<object> - items; items 1-100.

items[] members: description*: string - description; nonempty, max 300 chars; quantity*: number - quantity; >0; unit_price*: number - unit price; >=0.

Reference: skills/quotation-package/skill.json (#/input_schema, #/example). GET /api/skills/quotation-package.

responsibility-matrix | Responsibility matrix

Assign responsible, accountable, consulted and informed colleagues for each task and export a RACI workbook. Effect: local.

Parameters: project*: string - Project name; nonempty, max 100 chars; assignments*: array<object> - Task responsibilities; items 1-100.

assignments[] members: task*: string - task; nonempty, max 150 chars; responsible*: array<string> - responsible; items 1-20; accountable*: string - accountable; nonempty; consulted*: array<string> - consulted; items 0-20; informed*: array<string> - informed; items 0-20.

Reference: skills/responsibility-matrix/skill.json (#/input_schema, #/example). GET /api/skills/responsibility-matrix.

Sources: skills/procurement-compare/skill.json | skills/project-plan/skill.json | skills/proposal-package/skill.json | skills/quotation-package/skill.json | skills/responsibility-matrix/skill.json

Skill reference 51-55

risk-register | Risk register

Rank risks by likelihood and impact, preserving owners and mitigations. Effect: local.

Parameters: risks*: array<object> - Risks; items 0-500.

risks[] members: risk*: string - Risk; nonempty; likelihood*: integer - Likelihood, 1 to 5; >=1, <=5; impact*: integer - Impact, 1 to 5; >=1, <=5; owner*: string - Owner; nonempty; mitigation*: string - Mitigation; nonempty.

Reference: skills/risk-register/skill.json (#/input_schema, #/example). GET /api/skills/risk-register.

scenario-compare | Scenario comparison

Score options across weighted cost, quality, time or other numeric criteria with a visible breakdown. Effect: local.

Parameters: criteria*: array<object> - Decision criteria; items 1-20; options*: array<object> - Options and measures; items 2-30.

criteria[] members: name*: string - name; nonempty, max 100 chars; weight*: number - weight; >=0, <=100; direction*: string - direction; one of lower/higher.

options[] members: name*: string - name; nonempty, max 100 chars; measures*: object - Numeric measures by criterion name; values number.

Reference: skills/scenario-compare/skill.json (#/input_schema, #/example). GET /api/skills/scenario-compare.

schedule-builder | Schedule Builder

Run the existing synthetic tourism schedule builder stage. Effect: local.

inquiry*: object with travel_start*: ISO date and duration_days*: positive whole value. activities: array<object> (default []). request_id: string - trace identifier.

activities[]: id, name and duration_minutes; optional preferred_slot (morning, afternoon or evening; default afternoon) and source. Slots are limited to four hours; unplaced activities are reported.

flight: optional object with depart_date and return_date matching the itinerary. The output spans duration_days + 1 dates, including arrival and departure, in Asia/Shanghai.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/schedule-builder/skill.json (#/input_schema, #/example). GET /api/skills/schedule-builder.

shareholder-report | Shareholder report package

Calculate key financial results from supplied figures and create a draft Word report and PowerPoint shareholder update. Effect: local.

Parameters: period*: string - period; nonempty, max 100 chars; currency*: string - currency; min 3 chars, max 3 chars; revenue*: number - revenue; >=0; prior_revenue*: number - prior revenue; >=0; cost_of_sales*: number - cost of sales; >=0; operating_expenses*: number - operating expenses; >=0; cash_start*: number - cash start; cash_end*: number - cash end; highlights*: array<string> - highlights; items 1-6; risks*: array<string> - risks; items 1-6; actions*: array<string> - actions; items 1-6.

Reference: skills/shareholder-report/skill.json (#/input_schema, #/example). GET /api/skills/shareholder-report.

sheets-create | Create Google spreadsheet

Create a new Google spreadsheet with supplied headers and rows. Effect: remote_write.

Parameters: title*: string - Title; nonempty; values*: array<array<string/number/boolean/null>> - Rows; items 0-1000.

Reference: skills/sheets-create/skill.json (#/input_schema, #/example). GET /api/skills/sheets-create.

Sources: skills/risk-register/skill.json | skills/scenario-compare/skill.json | skills/schedule-builder/skill.json | skills/shareholder-report/skill.json | skills/sheets-create/skill.json

Skill reference 56-60

sheets-read | Read Google Sheets

Read values from an explicit spreadsheet and A1 range. Effect: remote_read.

Parameters: spreadsheet_id*: string - Spreadsheet ID; nonempty; range*: string - A1 range; nonempty.

Reference: skills/sheets-read/skill.json (#/input_schema, #/example). GET /api/skills/sheets-read.

sheets-write | Write Google Sheets

Write supplied values into an explicit spreadsheet range using RAW input. Effect: remote_write.

Parameters: spreadsheet_id*: string - Spreadsheet ID; nonempty; range*: string - A1 range; nonempty; values*: array<array<string/number/boolean/null>> - Rows; items 0-1000.

Reference: skills/sheets-write/skill.json (#/input_schema, #/example). GET /api/skills/sheets-write.

slides-create | Create Google Slides

Create an editable Google Slides presentation with titles and text. Effect: remote_write.

Parameters: title*: string - Presentation title; nonempty; slides*: array<object> - Slides; items 0-30.

slides[] members: title*: string - Slide title; nonempty; bullets*: array<string> - Key points; items 0-6.

Reference: skills/slides-create/skill.json (#/input_schema, #/example). GET /api/skills/slides-create.

spreadsheet-analyze | Analyze a spreadsheet

Profile an uploaded Excel or CSV file, including missing values and numeric totals. Effect: local.

Parameters: file_id*: string - Uploaded file ID; nonempty.

Reference: skills/spreadsheet-analyze/skill.json (#/input_schema, #/example). GET /api/skills/spreadsheet-analyze.

spreadsheet-create | Excel workbook

Create a styled Excel workbook from columns and rows. Effect: local.

Parameters: title*: string - Workbook title; nonempty; columns*: array<string> - Columns; items 0-50; rows*: array<array<string/number/boolean/null>> - Rows; items 0-5000.

Reference: skills/spreadsheet-create/skill.json (#/input_schema, #/example). GET /api/skills/spreadsheet-create.

Sources: skills/sheets-read/skill.json | skills/sheets-write/skill.json | skills/slides-create/skill.json | skills/spreadsheet-analyze/skill.json | skills/spreadsheet-create/skill.json

Skill reference 61-65

spreadsheet-edit | Edit a reviewed Excel input

Change one non-formula cell in an Excel workbook after review; preserve the original and produce a new version. Effect: reviewed_write.

Parameters: file_id*: string - Workspace Excel file ID; nonempty; sheet*: string - Worksheet; nonempty; cell*: string - Exact input cell; pattern ^[A-Za-z]{1,3}[1-9][0-9]{0,4}\$; value*: string/number - New number or text.

Reference: skills/spreadsheet-edit/skill.json (#/input_schema, #/example). GET /api/skills/spreadsheet-edit.

spreadsheet-search | Search an Excel workbook

Search every worksheet for a category, customer, reference or value and return exact cell locations. Effect: local.

Parameters: file_id*: string - Workspace Excel file ID; nonempty; query*: string - Search text; min 2 chars, max 100 chars.

Reference: skills/spreadsheet-search/skill.json (#/input_schema, #/example). GET /api/skills/spreadsheet-search.

support-triage | Support triage

Prioritize a ticket using customer impact and urgency and record its owner. Effect: local.

Parameters: title*: string - Issue; nonempty; impact*: string - Impact; one of low/medium/high; urgency*: string - Urgency; one of low/medium/high; owner*: string - Owner; nonempty; details*: string - Details; nonempty.

Reference: skills/support-triage/skill.json (#/input_schema, #/example). GET /api/skills/support-triage.

task-tracker | Task tracker

Save an owned task with status and due date to the local work register. Effect: local.

Parameters: title*: string - Task; nonempty; owner*: string - Owner; nonempty; due*: string - Due date; nonempty; status*: string - Status; one of todo/in_progress/blocked/done.

Reference: skills/task-tracker/skill.json (#/input_schema, #/example). GET /api/skills/task-tracker.

team-capacity | Team capacity

Inspect current staff capacity, availability and skill coverage. Effect: local.

Parameters: No parameters; supply {}.

Reference: skills/team-capacity/skill.json (#/input_schema, #/example). GET /api/skills/team-capacity.

Sources: skills/spreadsheet-edit/skill.json | skills/spreadsheet-search/skill.json | skills/support-triage/skill.json | skills/task-tracker/skill.json | skills/team-capacity/skill.json

Skill reference 66-69

telegram-boss-update | Notify boss on Telegram

After a completed task and explicit review, send its recorded result to the configured boss Telegram chat. Effect: remote_write.

Parameters: run_id*: string - run id; nonempty; note*: string - note; max 500 chars.

Reference: skills/telegram-boss-update/skill.json (#/input_schema, #/example). GET /api/skills/telegram-boss-update.

tour-search | Tour Search

Run the existing synthetic tourism tour search stage. Effect: local.

inquiry*: object; inquiry.group_size*: positive whole value. inquiry.interests: array<string> (default []); inquiry.accessible: boolean (default false). request_id: string - trace identifier.

catalog: object - optional replacement for the bundled activity fixture; activities must be an array. Optional source identifies provenance.

catalog.activities[]: id, name, max_group, accessible, tags, price_cny_per_person, preferred_slot, duration_minutes. Capacity and accessibility filter candidates; interest matches and group price determine ranking. Full fixture: resources/tour-catalog.example.json.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/tour-search/skill.json (#/input_schema, #/example). GET /api/skills/tour-search.

workload-rebalance | Workload rebalance

Split a set of tasks across qualified colleagues using projected capacity and identify work that needs a manager. Effect: local.

Parameters: tasks*: array<object> - Tasks in priority order; items 1-100.

tasks[] members: task*: string - task; nonempty, max 150 chars; required_skill*: string - required skill; nonempty, max 90 chars; hours*: number - hours; >0, <=168; preferred_person: string - preferred person; max 100 chars.

Reference: skills/workload-rebalance/skill.json (#/input_schema, #/example). GET /api/skills/workload-rebalance.

workload-splitter | Workload Splitter

Run the existing synthetic tourism workload splitter stage. Effect: local.

inquiry*: object with travel_start*: ISO date and budget_cny*: positive whole value. option*: object with quote_cny*: positive whole value; supplier_estimate_cny and transport_estimate_cny optionally support margin calculation.

hierarchy*: object; assignments maps duty names to owner IDs. routes optionally maps duty names to reporting paths. request_id: string - trace identifier.

Pass option-comparison.data.recommended and hierarchy-router.data. Unassigned work and past due dates escalate. The executive approval task is added for quote_cny >= 100000.

Schema note: the manifest accepts an object; detailed business validation is in tourism_core.py.

Reference: skills/workload-splitter/skill.json (#/input_schema, #/example). GET /api/skills/workload-splitter.

Sources: skills/telegram-boss-update/skill.json | skills/tour-search/skill.json | skills/workload-rebalance/skill.json | skills/workload-splitter/skill.json